

Final Report

In our final solution, we stuck with our context based treatment approach. Whatever is mentioned in this report is only for one context. The dataset was first split context wise and then each context was used to train a separate model which specialises in that context.

The final solution we used, differs from our mid term plan in two major ways.

Firstly, we noticed a flaw with our dataset. The first obvious issue was that it was small, as in the number of entries per disease were too less but that would have been still manageable especially for a prototype model. The bigger issue was that in the response taken column (which we were using as our column) they had somehow written the exact same response for every particular disease. So all our decision tree based model had to do was memorize the response taken just by looking at the disease name.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Patient ID			Primary Disease		Diseases / Diagnoses		Systolic Bf	Diastolic E	SpO2	Heart Rate	Respiratory Rate		first_drug_given				
2	10006			Sepsis		Sepsis; Infection and		91	50	98	104	25		.9% Normal Saline				
3	10006			Sepsis		Sepsis; Infection and		91	50	98	99	17		.9% Normal Saline				
4	10006			Sepsis		Sepsis; Infection and		111	60	98	96	20		.9% Normal Saline				
5	10006			Sepsis		Sepsis; Infection and		114	46	98	95	23		.9% Normal Saline				
6	10006			Sepsis		Sepsis; Infection and		117	51	97	92	20		.9% Normal Saline				
7	10006			Sepsis		Sepsis; Infection and		120	54	98	88	20		.9% Normal Saline				
8	10006			Sepsis		Sepsis; Infection and		119	51	99	92	19		.9% Normal Saline				
9	10006			Sepsis		Sepsis; Infection and		121	45	97	92	25		.9% Normal Saline				
10	10006			Sepsis		Sepsis; Infection and		124	50	100	93	17		.9% Normal Saline				
11	10006			Sepsis		Sepsis; Infection and		132	55	99	80	26		.9% Normal Saline				
12	10006			Sepsis		Sepsis; Infection and		117	50	100	76	22		.9% Normal Saline				
13	10006			Sepsis		Sepsis; Infection and		128	57	100	74	15		.9% Normal Saline				
14	10006			Sepsis		Sepsis; Infection and		120.5	51	98	70	22		.9% Normal Saline				
15	10006			Sepsis		Sepsis; Infection and		121	51	98	71	20		.9% Normal Saline				
16	10006			Sepsis		Sepsis; Infection and		116	48	99	76	17		.9% Normal Saline				
17	10006			Sepsis		Sepsis; Infection and		120.5	51	99	79	26		.9% Normal Saline				
18	10006			Sepsis		Sepsis; Infection and		141	58	97	87	20		.9% Normal Saline				
19	10006			Sepsis		Sepsis; Infection and		140	58	97	81	21		.9% Normal Saline				
20	10006			Sepsis		Sepsis; Infection and		120.5	51	99	74	20		.9% Normal Saline				
21	10006			Sepsis		Sepsis; Infection and		150	51	98	73	17		.9% Normal Saline				
22	10006			Sepsis		Sepsis; Infection and		150	51	98	67	19		.9% Normal Saline				
23	10006			Sepsis		Sepsis; Infection and		120.5	51	98	66	14		.9% Normal Saline				
24																		

This would have given a high accuracy when it was being tested on our own dataset (around 99 percent),

```
... Choose Files No file chosen Upload widget is only available when the
Saving UpdatedDataset.xlsx to UpdatedDataset.xlsx

--- Model Accuracy ---
Training Accuracy : 100.00%
Testing Accuracy : 100.00%

--- Test Set Classification Report ---
              precision    recall  f1-score   support

0               1.00        1.00        1.00         5

 accuracy               1.00        1.00        1.00         5
 macro avg              1.00        1.00        1.00         5
 weighted avg           1.00        1.00        1.00         5

--- Test Set Confusion Matrix ---
```

but would have flopped disastrously in real life because in a real scenario doctor's won't give the same dose of the same medicine for a particular disease irrespective of what the patient conditions are at that moment. The fact that the dataset had this must have been a mistake.

So we got another dataset from Hugging Face, which had the exact same structure, except for the fact the response column was properly maintained. Different vital parameters had different responses even for the same disease.

L	M	N	O	P	Q	R	S	T	U	V	W	X
icd_codes	vital_data	temp	spo2_pct	heart_rate	respiratory	systolic_bp	diastolic_bp	mean_bp	minutes_v	drug_name	dose	
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	ringers sol	1,000 ml		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	insulin glar	15 units		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	lasix	40 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	ondansetr	4 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	famotidine	20 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	colace	100 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	sodium ch	1 ml		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	nacl 0.9%	pyxis		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	acetamin	975 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	potassium	40 meq		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	aspirin	81 81 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	1000 ml : li	1,000 mU/hr		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	sodium ch	500 ml		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	50 ml syrin	25 4		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	midazolam	0 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	hydromor	1 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	473 ml - c	15 1		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	37.5 g tube	15 g		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	enoxaparir	30 mg		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	dextrose	5 25 gm		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	15 ml cup : 20-60 meq			
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	potassium	15 mmol		
995.92, r65.20	288.8, d72.829	426.0, i44.2	518.81, j96.00	787.91, r19.7	436, i67.8	572.2	584.9, n17.9	785.52, r65	glucose	15 30 gm		

So the first change we made was change our dataset.

After this, when we tried to train our models on this new dataset, we ran into a new problem. This time our dataset was again small and the way we set up our model was that the response taken column was our target

value. So we were basically making a multi class classifier. But the classes had unequal representation. They were so unbalanced that some classes (responses) occur only once in the entire dataset.

The model was unable to be trained properly on this dataset and was giving a very low accuracy (around 66 percent in training and 7 percent testing).

```
• --- OUTPUT 1 (`drug_name`) ACCURACY ---  
  Training Accuracy : 66.17%  
  Testing Accuracy  : 7.84%  
  
  --- OUTPUT 2 (`dose`) ACCURACY ---  
  Training Accuracy : 72.14%  
  Testing Accuracy  : 2.08%
```

Changing the model parameters had no effect as the problem was much more fundamental.

```
model1 = XGBClassifier(  
    n_estimators=400,  
    max_depth=400,  
    learning_rate=0.05,  
    objective='multi:softprob',  
    subsample=0.8,  
    colsample_bytree=0.8,  
    random_state=42,  
    eval_metric='mlogloss'  
)
```

To solve this problem, we took an assumption. Since our primary job was to detect when to take a response looking at the patient vitals, rather than suggesting exactly what response needs to be taken, we reduced the problem to one of binary classification. We trained the model to only see if there was a response or not in the response column. If there was a response, the model was expected to learn that under those combination of parameters, it was supposed to give a response. And in case the response column was blank, it was expected to learn that those vitals represented stable condition and that no response was needed. This is where the assumption comes into play. Since almost all the rows in the response column had an entry, we assumed that any set of parameters which were recorded in the dataset were worthy of a response and anything outside of this represents a stable patient (with respect to that specific context only). This would have worked better in a larger dataset.

We also added risk factor prediction, where if the model feels a response is necessary, it also learns and predicts exactly how much danger the patient is in on a scale of 5.

One obvious drawback is that we don't have anything left to test our results on, since we are using the entire dataset for the training.

To generate the final test results, we used the same method that was mentioned in the mid term report.

After multiple tests with decision tree based models like XG Boost, Random Forest and Light GBM, we found that XG Boost provided the best results.